

Vehicle ReID: документация решения

ASU Team · ЛЦТ 2026 · Состояние на 28 сентября 2026 года Профиль: MVP_fusion_v25.
Документ описывает подготовленную версию решения.

Задача и сценарий

На вход поступают снимок автомобиля и его ограничивающий прямоугольник, BBox. Нужно найти тот же автомобиль среди снимков из других локаций, не используя государственный номер. В gallery может не быть подходящего автомобиля, поэтому система умеет отказаться от совпадения.

Пользователь выбирает запрос или загружает JPEG/PNG, задаёт BBox и запускает поиск. Интерфейс показывает похожие автомобили, их кропы и оценку близости. Принятый кандидат отображается отдельно. Если уверенности не хватает, пользователь видит отказ и может просмотреть выдачу.

Для конкурсного расчёта тот же метод доступен через пакетный запуск: два CSV и изображения преобразуются в три выходных файла. Подготовленная версия содержит исходники, готовые модели и рецепты повторного обучения.

Данные и подготовка изображения

В каждом входном CSV нужны поля `image_id, x, y, w, h`. Координаты задаются в пикселях; ширина и высота положительны. Имена файлов соответствуют `image_id`; поддерживаются JPG/JPEG/PNG. Query и gallery имеют уникальные непересекающиеся ID. Для полной десятки нужны хотя бы десять объектов gallery.

Приложение проверяет изображение, приводит его к RGB с учётом EXIF и вырезает предоставленный BBox. Выходящий за границы или некорректный BBox, отсутствующий файл и ошибка декодирования останавливают расчёт. Объекты не пропускаются молча.

Кроп растягивается до квадратного входа модели: 208×208 для legacy и 256×256 для трёх R1. Точные преобразования, нормировка и контрольные суммы зафиксированы в конфигурации моделей. В активном inference нет поиска или дополнительного маскирования номера.

ID связывает результат с исходной строкой. Камера, время, география и метка автомобиля не поступают энкодеру или ранжировщику. Другие query не используются для обработки текущего запроса.

Как устроена модель

Основой служит OSNet-AIN x1.0. Готовый профиль использует четыре ONNX-модели:

Компонент	Вход	Выход	Роль
Legacy HPO, эпоха 5	208×208	512	Первый блок признаков
R1, seed 20260915	256×256	512	Участник ансамбля R1
R1, seed 20260916	256×256	512	Участник ансамбля R1
R1, seed 20260917	256×256	512	Участник ансамбля R1

Три R1-вектора нормируются и объединяются с равными весами в блок размерности 1536. Вместе с нормированным legacy-блоком получается вектор из 2048 чисел. Оба блока имеют единичную норму; весь экспортируемый вектор имеет норму около $\sqrt{2}$. Общая L2-нормализация к нему не применяется.

Ранжирование

Исходная близость состоит из равных долей cosine legacy и cosine ансамбля R1. Затем потоковый k-reciprocal reranking уточняет порядок с параметрами $k_1=20$, $k_2=3$, $\lambda=0.5$.

Граф соседства строится только по статичной gallery. Каждый query обрабатывается отдельно. Состав и порядок остальных query не должны менять его результат.

Совпадение и отказ

Кандидата выбирает отдельная ветвь: максимальный raw cosine ансамбля R1. Совпадение принимается при $\text{confidence} \geq 0.534365177154541$. В противном случае система отказывается.

confidence выражает близость признаков, а не вероятность правильного ответа. Кандидат R1 может отличаться от первого автомобиля в итоговом ранжировании. Это предусмотрено моделью: порядок десятки и решение об отказе решают разные задачи.

Порог выбран на calibration по $0.7 \times F1 + 0.3 \times \text{TNR}$; при равенстве учитываются F1, затем больший порог. Он сохранён вместе с моделью и не подбирается на входном тесте. Ручной порог в интерфейсе относится только к демонстрации.

Архитектура приложения

React-интерфейс обращается к FastAPI. Общий runtime загружает ONNX-модели, извлекает признаки и выполняет поиск. В веб-приложении PostgreSQL 16 с pgvector хранит gallery и её признаки. Для текущей gallery используется полный точный поиск, без приближённого индекса.

Пакетный процесс загружает gallery в память и работает без PostgreSQL, миграций и веб-сервера. API и batch используют общую реализацию v25. Это позволяет сопоставлять результаты интерфейса с конкурсными файлами.

Изображение + BBox

- проверка и кроп
- legacy + три R1
- два блока признаков, 2048D
- ranking: смесь 50/50 и k-reciprocal
- candidate: raw R1 и фиксированный порог
- интерфейс / API / файлы результатов

При запуске проверяются конфигурации и хеши моделей. Кэш gallery учитывает данные, BBox, модель и execution provider. Несовместимые признаки не должны использоваться повторно. Карта модулей приведена в ARCHITECTURE.md (ARCHITECTURE.md).

Запуск

Нужны Docker и Compose v2. Данные организаторов передаются отдельно. На машине с NVIDIA дополнительно требуется рабочая поддержка GPU в Docker. Окружение закреплено в requirements.txt, requirements-gpu.txt, frontend lock-файле и Docker-конфигурации.

После сборки образа:

```
docker compose --profile inference run --rm --no-deps --pull never inference \
  --dataset /data --output /out/run1
```

Каталог dataset/ монтируется только для чтения; результат записывается в artifacts/run1/. Этот процесс запускается без сети. Путь к данным меняется через HOST_DATASET_DIR, к выходному корню через OUTPUT_DIR в .env.

Полная последовательность сборки, CPU/CUDA-запуск, веб-демо и проверка результата: RUNBOOK.md (RUNBOOK.md). Для переноса готовых образов на закрытый стенд: CONTEST_IMAGE_DELIVERY.md (CONTEST_IMAGE_DELIVERY.md).

Выходные файлы

Файл	Содержимое
submission.csv	Без заголовка. Query ID и ровно десять различных gallery ID на строку. Все query в исходном порядке, включая отказы.
candidates.csv	Заголовок query_id,gallery_id,confidence. Один принятый R1-кандидат на запрос. При отказе строки нет.
embeddings.npy	float32, размер (N_query + N_gallery, 2048). Сначала query, затем gallery, в порядке входных CSV.
export_manifest.json	Порядок ID, профиль, хеши входов, моделей и кода, проверка форматов.
runtime_timing.json	Время загрузки и полного экспорта. Не подменяет отдельный замер скорости извлечения признаков.

В подготовленном наборе 1110 query и 750 gallery: embeddings.npy имеет размер (1860, 2048), submission.csv содержит 1110 строк, candidates.csv содержит 1069 принятых запросов. Для 41 запроса записан отказ через отсутствие строки.

Файлы доступны в submission/MVP_fusion_v25 (../submission/MVP_fusion_v25/README.md). Паспорт связывает их с runtime-коммитом 91d559c2e0024fd117f74943781b6b9ee84c997d и точными весами. Изменения документации не меняют версию использованной модели.

API

Локальная интерактивная схема доступна по /docs; Swagger-ресурсы включены в образ.

Метод и путь	Назначение
GET /api/health	Готовность, активный профиль, provider, размерность и размер gallery
GET /api/queries	Список подготовленных запросов
POST /api/search/query	Поиск по query ID из датасета
POST /api/search	Поиск по загруженному изображению и BBox
POST /api/embedding	Извлечение 2048-мерного вектора
GET /api/metrics	Сохранённые метрики development-validation
GET /api/images/{split}/{image_id}	Исходное изображение или кроп для просмотра

Загрузка принимает JPEG/PNG до 15 МиБ и 25 мегапикселей. Ответ поиска содержит ранжированные результаты, confidence, признак refused и accepted_candidate. В режиме кандидатов при отказе accepted_candidate=null. Поля и допустимые параметры описаны в OpenAPI приложения.

Обучение и воспроизведение

Активные четыре модели дообучены на размеченной части данных организаторов. Разбиение содержит 925 train, 307 calibration и 309 validation identity. В обучающую часть входят 5717 изображений. Для точных дубликатов исходных кадров проверено отсутствие пересечений между частями. Выданные тестовые изображения не участвуют в обучении и подборе параметров.

Инициализация: публичная OSNet-AIN x1.0, OpenVINO Open Model Zoo vehicle-reid-0001, версия 2022.1. Источник, лицензия MIT и хеши сохранены в models/README.md (../models/README.md) и ASSET_PROVENANCE.json (ASSET_PROVENANCE.json). Внешний датасет NiVe и модели CLIP/TransReID в активный v25 не входят.

В reproduction-kit (../reproduction-kit/README.md) находятся исторические исходники, разбиения, параметры и команды обучения. Legacy останавливается на пятой эпохе при расписании learning rate на 30 эпох. R1 останавливаются на шаге 800 при расписании на 1700 шагов. Замена горизонта на число выполненных шагов меняет рецепт.

Комплект проверяет хеши данных и исходников. Повторный экспорт трёх сохранённых R1-checkpoint дал побайтово те же ONNX. У legacy совпали тензоры и проверенные численные выходы; различие имени узла в графе изменило хеш файла. Полное повторное обучение в ходе подготовки комплекта не выполнялось. Подробности и команды: VALIDATION.md (../reproduction-kit/VALIDATION.md).

Обучающие исходники рассчитаны на macOS/Linux и Python 3.11.9. Исторический код использует fcntl. Численная и тем более побайтовая идентичность нового обучения на другом устройстве заранее не обещается.

Качество и проверки

Сохранённая development-validation: 309 query, 896 gallery; у 247 запросов есть допустимое совпадение, у 62 его нет. Основная метрика mAP@10 оценивает порядок результатов для запросов с совпадениями. F1 и TNR описывают выбранного кандидата и отказ.

Система	mAP@10	Rank-1	F1	TNR
Legacy OSNet с reranking	81,47%	80,16%	72,86%	79,03%
v24: legacy ranking + R1 candidate	81,47%	80,16%	79,55%	70,97%
v25: смесь legacy/R1 + R1 candidate	82,90%	83,00%	79,55%	70,97%

Прирост mAP@10 v25 к legacy составляет 1,43 процентного пункта. Рост F1 сопровождается снижением TNR: модель чаще принимает запросы, в том числе ошибочно. Поэтому одну метрику нельзя выдавать за улучшение всех сторон качества.

Выбор ranking v25 сделан на calibration из 84 заранее заданных конфигураций. Исторический legacy выбирался с использованием validation, а разбиения многократно наблюдались в ходе исследований. Приведённые цифры описывают разработку; оценку на скрытом тесте дадут организаторы.

Основания: V25_QUALITY_EVIDENCE.json (V25_QUALITY_EVIDENCE.json), метрики профиля (../models/MVP_fusion_v25.metrics.json), история экспериментов (EXPERIMENT_HISTORY.md).

При подготовке тестовых файлов проверены входные хеши и форматы; поиск повторён по сохранённым векторам для всех 1110 запросов. Тор-10, кандидат, confidence и отказ совпали. Шесть векторов заново получены из изображений с нулевой максимальной ошибкой относительно NPY. Это проверка согласованности экспорта, а не оценка качества теста.

Ограничения

- Скорость полного извлечения четырьмя моделями на конкурсном оборудовании в этом документе не подтверждается. CPU-время экспорта не считается официальным GPU-бенчмарком.
- Маскировка номеров изучалась на отдельных диагностических выборках. Эти опыты не доказывают отсутствие остаточного номерного сигнала у v25 на всех данных.
- Junk-фильтрацию выполняет evaluator с разметкой. Экспорт содержит ровно десять ID по ответу №31 организаторов; вопрос о пополнении десятки после удаления junk в сохранённых разъяснениях остаётся неоднозначным.
- Поиск рассчитан и проверялся на конкурсной gallery. Масштабирование на миллионы объектов и промышленная эксплуатация не оценивались.
- Презентация и адрес размещённого прототипа готовятся отдельно. Этот документ не подтверждает, что все материалы уже опубликованы.

Требования сверены с таблицей ответов (../ORGANIZER_QA.md) и уточнениями чата от 28 сентября (ORGANIZER_CLARIFICATIONS_2026-09-28.md).

Запуск Vehicle ReID

Команды выполняются из корня репозитория в Bash, Zsh или терминале WSL. Профиль по умолчанию: MVP_fusion_v25.

1. Подготовить данные

Нужны Docker Engine или Docker Desktop с Compose v2. Сборка скачивает базовые образы и зависимости; модели уже находятся в репозитории.

```
dataset/
  images/          # JPG/JPEG/PNG, имя файла соответствует image_id
  test_query.csv    # image_id,x,y,w,h
  test_gallery.csv  # image_id,x,y,w,h
```

Изображения предоставляет организатор. Для поиска train.csv не нужен. Датасет не публикуется в Git.

По умолчанию используются ./dataset, ./artifacts и порт 8017. Другие пути и порт можно задать в .env:

```
HOST_DATASET_DIR=/absolute/path/to/dataset
OUTPUT_DIR=/absolute/path/to/results
PORT=8017
```

Для каждого полного экспорта выбирайте новую или пустую выходную папку. Существующие результаты не перезаписываются.

2. Получить конкурсные файлы на CPU

```
docker compose build inference
docker compose --profile inference run --rm --no-deps --pull never inference \
  --dataset /data --output /out/run1
```

В artifacts/run1/ появятся submission.csv, candidates.csv, embeddings.npy, манифест и замер полного экспорта. Batch работает без сети, PostgreSQL и веб-приложения. Входной каталог доступен только для чтения.

Проверка форматов готового экспорта:

```
docker compose --profile inference run --rm --no-deps --pull never \
  -e DATASET_DIR=/data --entrypoint python inference \
  -m backend.evaluate --validate-only --output /out/run1
```

Здесь backend.evaluate используется только как валидатор файлов. Его отдельный legacy-режим оценки не измеряет качество v25.

3. Запустить на NVIDIA

Нужен Linux x86_64 с доступной для Docker NVIDIA GPU либо Docker Desktop с WSL2 и GPU. Сначала проверьте доступность устройства в своей среде. GPU-конфигурация собирает образ linux/amd64 с зависимостями из requirements-gpu.txt.

```
dcgpu() { docker compose -f docker-compose.yml -f docker-compose.gpu.yml "$@"; }
dcgpu build inference
dcgpu --profile inference run --rm --no-deps --pull never --entrypoint python inference \
  -m backend.gpu --output /out/gpu/preflight.json
dcgpu --profile inference run --rm --no-deps --pull never inference \
  --dataset /data --output /out/gpu/run1
```

Preflight должен подтвердить preflight_passed: true, профиль MVP_fusion_v25, размерность 2048 и работу вычислительных операторов всех четырёх моделей на CUDA. При отсутствии нужного provider программа сообщает ошибку. Наличие GPU-конфигурации само по себе не подтверждает скорость на конкурсном стенде.

Для повторного экспорта используйте /out/gpu/run2. Сравнение сохранённых результатов:

```
dcgpu --profile inference run --rm --no-deps --pull never --entrypoint python inference \
-m backend.compare_exports --dataset /data \
--first /out/gpu/run1 --second /out/gpu/run2 \
--output /out/gpu/repeatability.json
```

Отдельный замер извлечения признаков, когда второй экспорт уже выполнен:

```
dcgpu --profile inference run --rm --no-deps --pull never --entrypoint python inference \
-m backend.benchmark --dataset /data --profile MVP_fusion_v25 \
--provider CUDAExecutionProvider --output /out/gpu/benchmark.json
```

Методика включает 50 прогревов, 300 измерений batch=1, median/p95 и throughput для batch 1/8/16/32. В измерение входят чтение, декодирование, кроп, preprocessing, все четыре модели и нормализация; поиск по gallery не входит. Эти команды описывают процедуру, а не уже полученный результат.

4. Открыть веб-приложение

CPU:

```
docker compose up -d --build
```

GPU, после определения dсgpi из предыдущего раздела:

```
dcgpu up -d --build
```

Откройте <http://127.0.0.1:8017>. Swagger доступен по <http://127.0.0.1:8017/docs>.

Первый запуск копирует датасет во внутренний volume, запускает PostgreSQL, применяет миграции и строит gallery. Понадобится дополнительное место примерно размером датасета. До завершения индексации сервис может быть не готов.

```
curl http://127.0.0.1:8017/api/health
```

Ожидаемые поля: status=ready, profile=MVP_fusion_v25, embedding_dim=2048, default_threshold=0.534365177154541. Для выданной gallery ожидается gallery_size=750. Provider должен соответствовать выбранному CPU/GPU-режиму.

Выберите готовый query или загрузите изображение, задайте VBox и выполните поиск. В режиме кандидатов отказ показывается отдельно от списка похожих машин. Ручная смена порога в демо не меняет экспорт с зафиксированным профилем.

Остановка без удаления данных: `docker compose stop` или `dcgpi stop`.

5. Запуск без интернета

Заранее соберите и перенесите образы по инструкции передачи Docker (CONTEST_IMAGE_DELIVERY.md). После `docker load`:

```
docker compose up -d --no-build --pull never
```

Для GPU используйте ту же команду через dсgpi. Веса и Swagger включены в образ. Batch-команды выше уже содержат `--pull never` и запускаются с `network_mode: none`.

6. Проверки разработчика

Изолированные тесты с временной PostgreSQL:

```
docker compose -p vehicle-reid-tests -f docker-compose.test.yml up --build --abort-on-container-exit \
--exit-code-from tests
docker compose -p vehicle-reid-tests -f docker-compose.test.yml down -v
```

Последняя команда удаляет ресурсы тестового проекта. Состав результатов и границы измерений описаны в документации (DOCUMENTATION.md).